Phase 12: Backend API

Hospital Patient Helpdesk Chatbot

Python module: `07_backend/12_api_main.py` **Jupyter notebook:** `13_notebooks/12_api_main.ipynb`
Purpose: Create a FastAPI backend with a `/chat` endpoint that returns guarded, citation-aware hospital helpdesk answers.

1. Phase objective

Phase 12 exposes the end-to-end RAG workflow through an API boundary. It composes:

1. Phase 10 RAG chain for retrieval, prompt construction, and generation.
2. Phase 11 guardrails for final emergency, diagnosis, dosage, citation, injection, and sensitive-data enforcement.
3. FastAPI route definitions for `/`, `/health`, and `/chat` when FastAPI dependencies are installed.

The reusable `ChatService` is independent of FastAPI. This lets notebooks and tests validate the same behavior even when the web framework is not installed in the active interpreter.

2. Folder placement

```
hospital_patient_helpdesk_chatbot/
|-- 01_data/
|   |-- sample_queries/
|   |   `-- test_questions.csv
|   `-- processed/
|       |-- 12_api_responses.json
|       |-- 12_api_report.json
|       |-- 12_api_audit.csv
|       |-- 12_failed_api_requests.json
|       `-- plots/
|           |-- 12_api_request_latency.png
|           `-- 12_api_safety_actions.png
|-- 02_config/
|   `-- prompt_config.yaml
|-- 05_vector_store/
|   `-- chroma_db/
|       `-- 06_vector_index.sqlite3
|-- 06_rag_pipeline/
|   |-- 10_rag_chain.py
|   `-- 11_safety_guardrails.py
|-- 07_backend/
|   |-- 12_api_main.py
|   |-- 12_api_main.md
|   |-- 12_api_main.pdf
|   |-- 13_request_schema.py
|   `-- 14_response_schema.py
|-- 12_tests/
|   `-- test_api.py
|-- 13_notebooks/
|   `-- 12_api_main.ipynb
|-- .env
`-- README.md
```

All Phase 12 generated outputs and plots use the 12_ prefix.

3. Runtime inputs

Input	Purpose
`05_vector_store/chroma_db/06_vector_index.sqlite3`	Searchable hospital evidence used by the live RAG chain.
`02_config/prompt_config.yaml`	Hospital assistant and safety prompt policy.
`.env`	Provider, model, and API-key configuration. The default provider is `offline`.

Input	Purpose
`01_data/sample_queries/test_questions.csv`	Synthetic requests for endpoint-level evaluation.
`06_rag_pipeline/10_rag_chain.py`	Retrieval, prompting, generation, source provenance, and timings.
`06_rag_pipeline/11_safety_guardrails.py`	Final safety and privacy policy enforcement.

The API evaluation does not use precomputed Phase 10 or Phase 11 JSON outputs. It runs the live service path so artifacts match `/chat` behavior.

4. Output files

Numbered artifact	Description
`12_api_responses.json`	Guarded response payloads produced by the chat service.
`12_api_report.json`	Request counts, mode counts, safety-action counts, latency summary, provider details, and dependency availability.
`12_api_audit.csv`	Privacy-conscious operational rows without storing question or answer text.
`12_failed_api_requests.json`	Sanitized failed request numbers and error classes.
`12_api_request_latency.png`	End-to-end chat-service latency by request.
`12_api_safety_actions.png`	Final guardrail action counts.

The plots are operational diagnostics and do not certify clinical safety.

5. API routes

`GET /`

Returns service metadata and the documentation path.

`GET /health`

Returns non-sensitive readiness metadata:

```
{
  "status": "ready",
  "service": "hospital-patient-helpdesk-chatbot",
  "api_version": "1.0",
  "provider": "offline",
  "model": "offline-grounded-v1",
  "index_ready": true,
  "guardrail_version": "1.0",
  "timestamp_utc": "2026-06-15T19:25:45Z"
}
```

`POST /chat`

Accepts a patient helpdesk question and returns a guarded answer. The route is synchronous because the local RAG pipeline is CPU and file based; FastAPI can still run sync endpoints safely in its normal execution model.

6. Request schema

```
{
  "question": "Where is the cardiology department?",
  "department": null,
  "content_category": null
}
```

Field	Rule
`question`	Required, whitespace-normalized, 2 to 1000 characters.

Field	Rule
`department`	Optional filter, max 100 characters.
`content_category`	Optional filter, max 100 characters.

Unknown fields are ignored by the framework-independent `ChatRequest` contract. FastAPI and Pydantic add JSON parsing, OpenAPI schema generation, and request-body validation when installed.

7. Response schema


```
{
  "request_id": "REQ-17E384FE9897",
  "answer": "Use the patient portal ... [S1]",
  "mode": "grounded_answer",
  "citations": ["[S1]"],
  "sources": [
    {
      "citation": "[S1]",
      "chunk_id": "faqs-hospital-faqs-json-0001-chunk-001",
      "source_file": "faqs/hospital_faqs.json",
      "source_type": "json",
      "department": "Portal Support",
      "content_category": "appointments",
      "page_reference": null,
      "score": 0.74580427
    }
  ],
  "retrieval_confidence": "high",
  "safety_flag": false,
  "guardrail_action": "pass",
  "risk_level": "low",
  "triggered_rules": [],
  "provider": "offline",
  "model": "offline-grounded-v1",
  "latency_ms": 5.665,
  "timestamp_utc": "2026-06-15T19:25:31Z"
}
```


Emergency, diagnosis, and dosage cases may have empty `citations` and `sources` because the returned answer is an approved safety message rather than source-derived content.

8. Python module code sections

Configuration and request contract

`ApiConfig` validates provider, host, port, and retrieval top-k settings. `ChatRequest` provides framework-independent request normalization and validation so tests and notebooks can exercise endpoint behavior without FastAPI installed.

Dependency loading

`import_module()` imports numbered project files by path. `load_pipeline_modules()` loads the Phase 10 chain and Phase 11 guardrails.

`ChatService`

`ChatService` is the main backend object. During initialization it loads pipeline modules, retrieval dependencies, vector index path, prompt policy, environment values, model config, chain config, and guardrail config. It validates readiness once rather than reloading data for every request.

`health()` returns a non-sensitive readiness summary.

`chat()` runs one request through Phase 10, applies Phase 11 guardrails, measures total latency, and returns the public response payload.

FastAPI adapter

`create_app()` imports FastAPI and Pydantic lazily. If the dependencies are missing, the shared service and offline evaluation still work, while app creation raises a clear installation message.

The app defines:

- Pydantic request and response models;
- `/`, `/health`, and `/chat` routes;
- no-store cache headers;
- `X-Content-Type-Options: nosniff`;
- `X-Frame-Options: DENY`; and
- sanitized error responses.

FastAPI docs referenced for this design: request bodies, response models, lifespan/app state patterns, and testing guidance are in the official FastAPI documentation.

API evaluation

`run_api_evaluation()` runs every synthetic question through `ChatService.chat()`, writes responses, audit rows, sanitized failures, report JSON, and both diagnostic plots. This checks the same behavior that `/chat` uses.

CLI

Run offline API-service evaluation:

```
python 07_backend/12_api_main.py
```

Start the server after installing dependencies:

```
python 07_backend/12_api_main.py --serve --host 127.0.0.1 --port 8000
```

Then use:

- `http://127.0.0.1:8000/docs`
- `GET http://127.0.0.1:8000/health`
- `POST http://127.0.0.1:8000/chat`

9. Notebook code sections

The notebook:

1. resolves the project root from the workspace, project, or notebook folder;
2. imports the shared backend module;
3. checks whether FastAPI is installed;
4. initializes `ChatService` with the offline provider;
5. verifies readiness through `health()`;
6. demonstrates request normalization and validation;
7. runs a grounded cardiology question;
8. runs an emergency question and verifies override behavior;
9. inspects the FastAPI OpenAPI contract when dependencies are present;
10. runs all 12 synthetic requests through the service;
11. validates report and response contracts; and
12. displays both numbered plots.

10. Notebook versus Python module

Topic	12_api_main.ipynb	12_api_main.py
Main purpose	Interactive backend walkthrough, examples, assertions, and plots.	Reusable API service, FastAPI app factory, CLI, and batch evaluation.
FastAPI dependency	Detects whether FastAPI is installed and continues service validation.	Creates FastAPI app when dependencies are installed; otherwise keeps service importable.
Business logic	Calls `ChatService` and `run_api_evaluation()`.	Owns service initialization, request validation, routes, headers, errors, artifacts, and plots.
Outputs	Displays examples and reports inline.	Writes `12_` JSON, CSV, and PNG artifacts.
Deployment	Not intended to serve traffic.	`--serve` starts Uvicorn after dependencies are installed.

The notebook does not duplicate API logic, so it remains aligned with the Python module.

11. Automated tests

`12_tests/test_api.py` covers:

- request normalization;
- invalid question rejection;
- service readiness;
- grounded chat response shape;
- emergency override response shape; and
- FastAPI OpenAPI routes when FastAPI and Pydantic are installed.

In the current interpreter, `pytest`, `fastapi`, `pydantic`, and `uvicorn` are not installed even though they are listed in `requirements.txt`. The module and notebook therefore validate the shared service path directly, and the FastAPI-specific test is dependency-gated.

12. Validation results

The offline API-service evaluation completed:

- 12 input requests;
- 12 responses created;
- 0 failed requests;
- 9 grounded answers;
- 1 emergency override;
- 2 unsafe-medical-advice overrides;
- 9 `pass` safety actions;
- 3 `override` safety actions; and
- generated request-latency and safety-action plots.

`12_api_report.json` records `fastapi_available: false` in the current environment. After installing dependencies from `requirements.txt`, `create_app()` and `--serve` will create the real FastAPI application.

13. Security, privacy, and deployment notes

- Use HTTPS through a trusted proxy or platform load balancer.
- Add authentication and authorization before exposing patient-facing routes.
- Restrict CORS to known frontends.

- Add rate limits and request-size limits.
- Use no-store caching for patient interactions.
- Avoid logging raw questions and answers unless privacy review permits it.
- Protect API keys and never return secrets in health or error payloads.
- Monitor latency, failures, safety overrides, and unexpected mode shifts.
- Keep the chatbot informational and clearly separate it from emergency care or clinical decision support.
- Re-run safety, privacy, and clinical review before deployment.

Official FastAPI references used: Request Body, Response Model, Lifespan Events, and Testing.